
Quantitative Forecasting of Sunspot Numbers Using Machine Learning

by

Tiasha Singha
Sirsha Dutta
Sanchari Sen

B.Sc Physics (Honours)

Supervisor: Dr. Suparna Roychowdhury



Xaverian Astronomical Society (XAS)
St. Xavier's College (Autonomous), Kolkata

This report is submitted in fulfilment of XAS undergraduate research project, 2024.

2024-25

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to our supervisor Dr. Suparna Roychowdhury, Deputy President of XAS, for guiding us in our project. We also extend our gratitude to Mr. Bappaditya Manna, Technical Officer of the Father Eugene Lafont Observatory, for hand-holding us through all our observation sessions, and allowing us to undertake discussions at FELO.

We are also thankful to our senior Subham Banerjee, for proposing the idea of this research, and helping us as we required. We thank our friends and family heartily for supporting us throughout this work. Lastly, we acknowledge all individual and organisation whose work has been cited in the making of this report.

This has been a collection of efforts of each individual of this team, and definately a milestone for us.

Signing off!
The Sunspot team

ABSTRACT

Sunspots are dark, cooler patches on the Sun’s surface caused by concentrated magnetic fields, appearing in roughly 11-year cycles and driving variations in solar activity that affect space weather.

In this project, we apply and compare three quantitative forecasting frameworks on 297 years of historical sunspot data (1700–1996) with out-of-sample validation on subsequent cycles (1997–2024). First, a Seasonal ARIMA (SARIMA) model leverages seasonal differencing and autoregressive–moving-average terms to capture the 11-year periodicity. Second, TBATS state-space models apply Box-Cox transformations, ARMA error terms, and damping factors to accommodate multiple seasonal effects. Third, an ensemble of thirty Multilayer Perceptrons (ANNs) is trained on overlapping 11-year sliding windows of Min–Max–scaled counts, producing point forecasts and 95% confidence intervals via inter-model percentiles.

By comparing classical and modern approaches—SARIMA, TBATS, and ANN—we highlight the complementary insights each method offers for anticipating solar-cycle progression. This systematic evaluation helps us with a diverse toolkit for planning and mitigating the impacts of space-weather events driven by sunspot activity.

Contents

Abstract	ii
1 Introduction	1
2 Observations and Prior Forecasting Methods	3
3 Predictive Modeling of Sunspot Activity	4
3.1 SARIMA	4
3.2 TBATS	5
3.3 Artificial Neural Networks (ANN)	6
3.4 Comparison of the three models	8
4 Future Prospects	10
A SARIMA code	11
B TBATS code	13
C ANN code	15

INTRODUCTION

Sunspots look like a ragged hole. It has a dark core- **umbra** in its interior and a less dark halo- **penumbra** as its exterior. The penumbra differs from the pores, which are smaller, dark features. Sunspots remain slightly depressed with respect to the surface and have a temperature $1500K$ below the surrounding temperature. They show great variation in size. The huge ones have diameters of 60000 km or more. The large ones or a tight group of small ones are visible to the naked eye under clement conditions. The small ones have diameters of around 3500 km and are more common. They also show varying lifetimes- from hours to months. According to the **Gnevyshev-Waldmeier** rule, most sunspots last less than a day. They are generally located in the active region of the Sun with bipolar magnetic structures. These active belts stretch up to 30° on each side of the solar equator. They are seen at higher latitudes during the early cycle, but with progress in the cycle the newer sunspots appear at lower latitudes. This behavior can be seen in *Butterfly Diagram*. The magnetic field strength of photosphere averaged over a sunspot is in the range $1800 - 3700\text{ G}$. The darkest part of umbra has $700 - 1000\text{ G}$ magnetic field strength while the outer edge of the penumbra has $700 - 1000\text{ G}$.

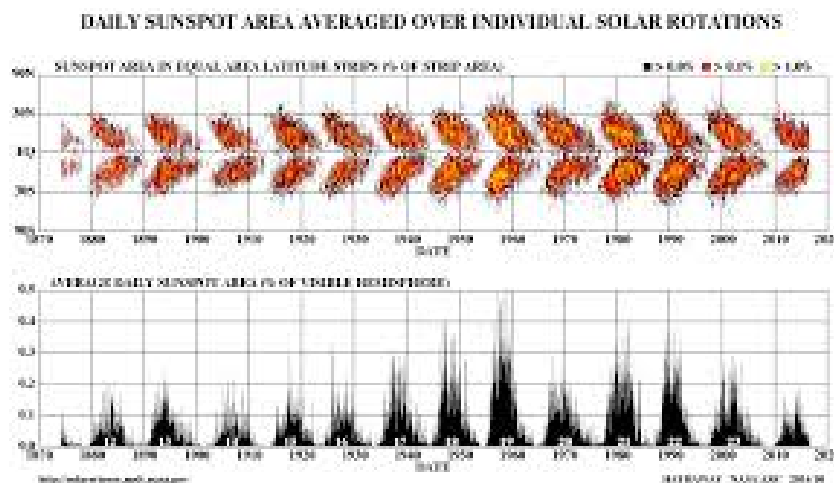


Figure 1.1: Butterfly Effect of sunspots[1]

The frequency of sunspots over a cycle is not constant. The **Solar Cycle**, also called **Sunspot cycle**, refers to the 11-year change in the activity of the Sun in terms of the number of measured sunspots on the surface. Over the period of a solar cycle, levels of solar radiation and ejection of solar material, the number and size of sunspots, solar flares, and coronal loops all exhibit a synchronized fluctuation from a period of minimum activity to a period of maximum activity back to a period of minimum activity. The

magnetic field of the Sun flips during each solar cycle, with the flip occurring when the solar cycle is near its maximum. After two solar cycles, the Sun's magnetic field returns to its original state, completing what is known as a *Hale cycle*. When the mean number of sunspots visible is tracked and plotted, it shows a periodic change from solar minimum to solar maximum and back to minimum over a rough period of 11 years.

Now, the solar cycles have an evident effect on space and Earth's atmosphere. During solar minimum, the polar coronal holes dominate with fast solar winds. And during solar maximum, low latitude holes dominate with slow winds. This affects the propagation of interplanetary disturbances and the background conditions they travel through. The magnetic field strength of the solar winds also peaks during the solar maximum, which affects Earth's magnetosphere, which leads to geomagnetic storms, auroras, and disruptions of satellite systems, radio communication and power grids. Thus, predicting the progression of the sunspot cycle could be beneficial and help in planning out the plans actions to prevent the mishaps and also study the extent of effects.



Figure 1.2: Image of Solar Disc with visible sunspots captured from **FELO**, using the *Meade Coronado Solar Telescope*

OBSERVATIONS AND PRIOR FORECASTING METHODS

In the 1600s, works of Galileo [2] and Christoph Scheiner claimed that the dark regions of the sun were indeed sunspots, and confirmed them through solar rotations and detailed diagrams. The 1800s saw consistent progress in the discovery of the 11-year solar cycle [3] and its link to geomagnetic storms, making it a concrete well established theory.

Initial studies in predicting the sunspot numbers were done in early 2000s, when Zhao et al, 2008 [4]. used Radial Basis Function in neural networks to predict monthly sunspot numbers. Ten years later, Abdel-Rahman and Marzouk [5] used Autoregressive Integrated Moving Average (ARIMA) models for prediction. The ARIMA models were found to be outperformed by both Pala and Atici's [6] work in 2019 using LSTM and Aggarwal et al. in 2020 using DNNs. Hybrid models have now come into the picture, SARIMA-SVM and CNN-LSTM, that combine properties of traditional time series forecast as well as neural networks.

There has been a shift from traditional methods of using time series analysis like ARIMA models to neural networks like DNN. Here is a breakdown of the feature difference between the two:

Time series models	Neural networks
Equation-based, works with stationary values and models temporal dependencies	Usually a black box architecture, the model learns through complex patterns in the data
Works with small datasets	Requires large amounts of data to train the model
Cannot account irregularities in the dataset very well	Trains the model through fluctuations and irregularities very well

Table 2.1: Comparison between time series models and neural networks

Observations are noted daily, and average of monthly and yearly data are calculated and documented. Data sources are available at ESO repositories, NASA repositories, etc. For our purpose, we use mean yearly data, available at [SILSO](#), [The Royal Observatory of Belgium](#).

PREDICTIVE MODELING OF SUNSPOT ACTIVITY

3.1 SARIMA

Sunspot numbers, recorded over centuries, display quasi- periodic behaviour aligned with the approximately 11-year solar cycle. Hence we use **Seasonal Autoregressive Integrated Moving Average (SARIMA)**, a time series forecasting model to predict sunspot activity for....

The model can be expressed as $SARIMA(p, d, q)(P, D, Q, S)$.

1. Non-seasonal parameters:

- p = Autoregressive (AR) order = No. of past observations that influence the current model
- d = Differencing order = Number of times the series is differenced to remove trend
- q = Moving Average (MA) order = No. of past forecast errors in the model

2. Seasonal parameters:

- P = Seasonal autoregressive (AR) order = No. of past observations that influence the current model
- D = Seasonal differencing order = Number of times the series is differenced to remove trend
- Q = Seasonal moving average (MA) order = No. of past forecast errors in the model
- S = Seasonal period length

We can write it in general form as

$$\Phi_P(B^S)\phi_p(B)(1-B)^d(1-B^S)^D y_t = \Theta_Q(B^S)\theta_q(B)\epsilon_t$$

where,

- Non seasonal AR of order p : $\theta_p(B) = 1 - \theta_1 B - \theta_2 B^2 - \dots - \theta_p B^p$
- Non seasonal MA of order q : $\theta_q(B) = 1 + \theta_1 B + \theta_2 B^2 + \dots + \theta_q B^q$
- Seasonal AR of order P : $\Phi_P(B^S) = 1 - \Phi_1 B^S - \Phi_2 B^{2S} - \dots - \Phi_P B^{PS}$
- Seasonal MA of order Q : $\Theta_Q(B^S) = 1 + \Theta_1 B^S + \Theta_2 B^{2S} + \dots + \Theta_Q B^{QS}$

For our case, we use SARIMA (1,1,1)(1,1,1,11), hence

$$(1 - \Phi_1 B^{11})(1 - \phi_1 B)(1 - B)(1 - B^{11})y_t = (1 + \Theta_1 B^{11})(1 + \theta_1 B)\epsilon_t$$

Training our model with the available sunspot data, and taking into account the 11 year seasonal sunspot cycle, we get the fit as follows:

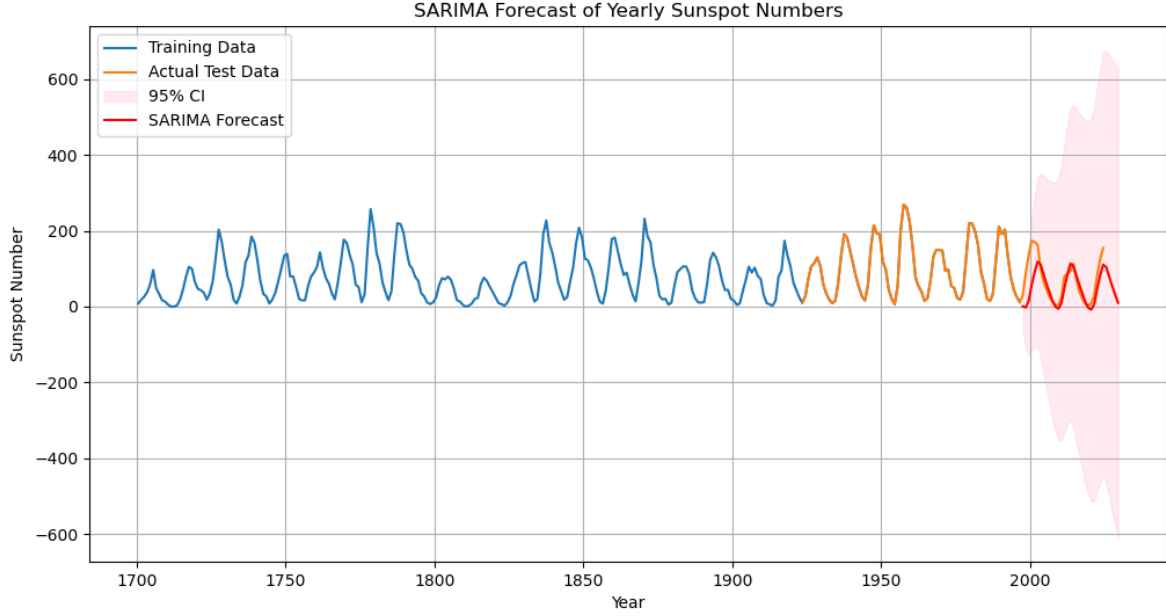


Figure 3.1: Comparison of observed sunspot numbers (1700-present) versus forecasts using SARIMA

Thus, we see that even though SARIMA provides a good enough fit initially, the confidence interval of this model is very high. This is because SARIMA, being a traditional time series model, underperforms when variations in the number of spots are encountered over a course of a few decades of data. To overcome this, we look at time series models with more features and neural networks.

3.2 TBATS

TBATS stands for **Trigonometric Box-Cox ARMA Trend Seasonal**. It was introduced by De Livera, Hyndman, and Snyder to address the limitations of models like ARIMA. Each of the term means the following:

- **Trigonometric Seasonal Representation**- It uses Fourier(trigonometric) terms to model the seasonality
- **Box-Cox Transformation**- It stabilizes the variance in data. It is given by -

$$y^\lambda = \begin{cases} \frac{y^\lambda - 1}{\lambda}, & \text{if } \lambda \neq 0. \\ \log y, & \text{if } \lambda = 0. \end{cases}$$

Here λ is the transformation parameter

- **AutoRegressive Moving Average (ARIMA)**- It captures the short-term autocorrelations in the residuals.
- **Trend Components**- It includes the random walk or constant mean and the trends.
- **Seasonal Components**- It uses Fourier series to handle multiple or changing seasonalities.

In a simplified form, it stands as:

$$y_t = l_t + b_t + \sum_{j=1}^J s_{j,t} + d_t + \epsilon_t$$

Here,

- l_t : Local Level
- b_t : Trend
- $s_{j,t}$: Seasonal components
- d_t : ARMA error component
- ϵ_t : White noise

The TBATS model is useful for analysing time-series data with multiple seasonal periods with changing seasonal patterns. Thus, after training the model with available data and taking into account the 11-year sunspot cycles, the following result is obtained:

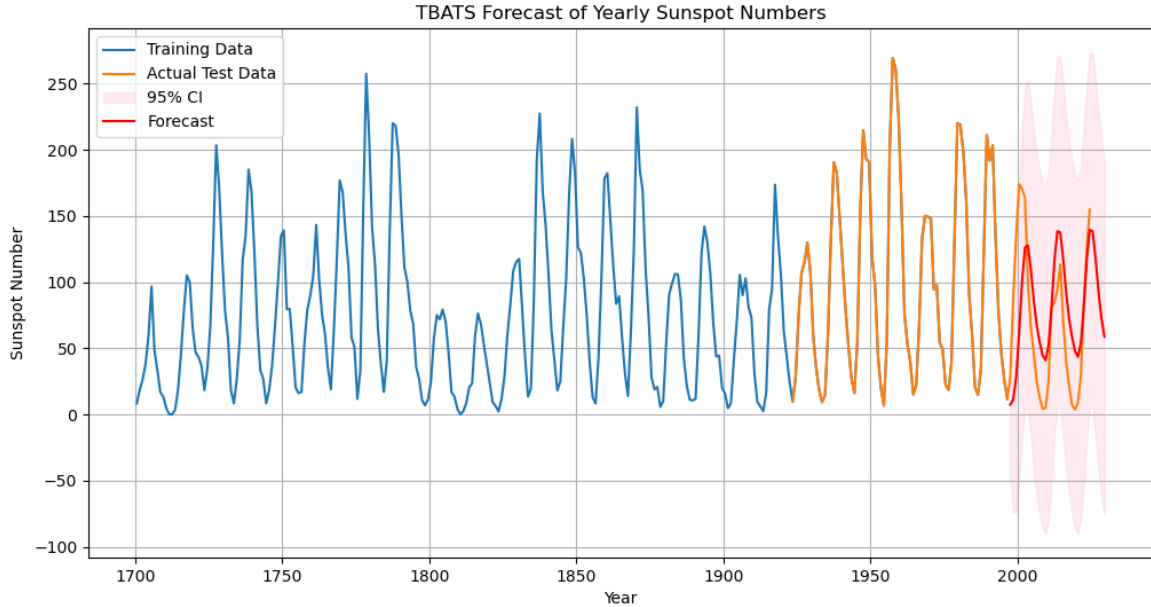


Figure 3.2: Comparison of observed sunspot numbers (1700-present) versus forecasts using TBATS

From the graph, it is seen that the predicted graph almost fits the original graph. The variation in the peaks shows that the model was able to take into account the non-fixed seasonal variability. However, there is an increase in the confidence interval (though less than SARIMA), implying that uncertainty increases with an increase in time interval. This reduces its dependability for long-term predictions.

3.3 Artificial Neural Networks (ANN)

Sunspots wax and wane in a roughly 11-year rhythm, yet the precise timing and strength of each cycle can vary substantially. Accurate forecasts matter for everything from satellite planning to electrical-grid resilience, since bursts of solar activity can drive geomagnetic storms that disrupt communications and power systems.

Traditional statistical methods (like TBATS) decompose the series into trend, multiple seasonal harmonics, and ARMA errors—often capturing sub-cycle bumps but producing very wide confidence bands far into the future. An alternative parametric approach is SARIMA, which explicitly models autoregressive, differencing, and seasonal (period = 11) terms—offering interpretable coefficients but demanding careful order selection and stationarity.

In contrast, Artificial Neural Network (ANN) is a machine-learning model inspired by biological brains. It “learns” patterns by adjusting weighted connections between layers of simple units (neurons), allowing it to approximate virtually any nonlinear relationship. It learns directly from historical patterns by fitting nonlinear transformations of their inputs. A simple feed-forward network—also called a multilayer perceptron—can model complex relationships between past sunspot numbers and future values without hand-crafting seasonal terms. By training an ensemble of such networks, we both stabilise point forecasts and quantify uncertainty via the spread of predictions.

1. Data & Pre-processing: We gathered 297 years of annual sunspot counts spanning 1700 through 1996 as our training set. Before feeding these values into the network, we applied a Min–Max scaling transformation to map every sunspot count into the 0,1 interval. This normalisation both accelerates convergence during training and prevents hidden-unit saturation by keeping inputs in a consistent numerical range.

2. Feature Engineering - Sliding-Window Sequences: To turn our time series into supervised learning examples, we slid an 11-year window across the scaled data. Each training example thus consists of the previous eleven years’ sunspot values as the feature vector, with the very next year’s count serving as the target. In effect, the network “reads” one full solar cycle’s history and learns to predict the subsequent point, capturing the characteristic rise-and-fall pattern.

3. Model Architecture & Training: We constructed an ensemble of thirty independent feed-forward neural nets using Scikit-Learn’s ‘MLPRegressor’. Each model shares the same basic design—one hidden layer with fifty ReLU-activated neurons and the Adam optimizer for up to 500 iterations—but differs in its random weight initialization (‘random_state’ set uniquely per model). Training each network on the sliding-window dataset encourages slight variations in learned weights, which our ensemble later leverages to quantify uncertainty.

4. Rolling-Forward Forecast: Once training finishes, we perform a recursive, rolling-forward forecast for the next 33 years. Starting from the final eleven years of known (scaled) data, each model predicts the first year of the forecast horizon, appends its own prediction to the input window, then repeats until all 33 future values are generated. This procedure ensures each prediction conditions on the network’s previous output, mirroring real-world forecasting where true future values are unknown.

5. Uncertainty Quantification: With thirty distinct model forecasts in hand for each of the 33 years, we compute the ensemble mean as our point estimate. To express forecast uncertainty, we extract the 2.5th and 97.5th percentiles across the thirty members at each horizon, forming a 95% confidence interval. This band reflects the variability introduced by different initializations and the network’s internal stochasticity.

6. Visualization: In the final plot, the full 1700–1996 historical series appears in blue, while the actual test data is overlaid in orange for direct comparison. The red curve traces the ANN ensemble’s mean forecast over the 33-year horizon, and the semi-transparent red shading highlights the 95% CI. This graph showcases how closely the model captures the solar-cycle amplitude and timing against observed values.

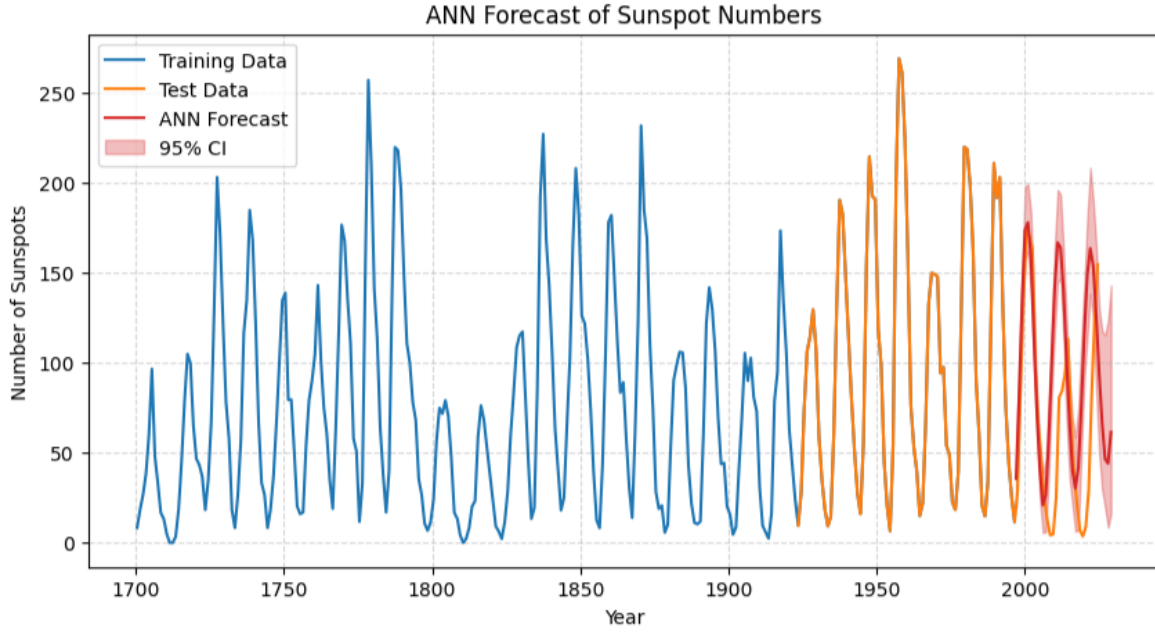


Figure 3.3: Comparison of observed sunspot numbers (1700-present) versus forecasts using ANN

Overall, the ensemble replicates the broad rise-and-fall of the solar cycle quite faithfully, typically peaking three to four years into each predicted cycle with amplitudes nearly matching the real data. The relatively narrow confidence band further suggests strong agreement among the thirty networks.

3.4 Comparison of the three models

Three forecasting methodologies—SARIMA, TBATS, and an artificial neural network (ANN)—were evaluated for predicting sunspot numbers using historical data spanning 1700 to the present. Evaluation Metrics measure the efficiency of a Machine Learning model. We have used three such evaluation metrics to not only test the accuracy of the individual models but also to compare the three and identify the most efficient one.

Mean Squared Error (MSE): The average of the squared differences between predicted and observed sunspot counts. By penalizing larger errors more heavily, MSE emphasizes overall forecast accuracy in terms of variance.

Mean Absolute Error (MAE): The average of the absolute differences between predicted and observed values. MAE provides an intuitive measure of typical forecast deviation, treating all errors equally.

Confidence Interval (CI): The degree of uncertainty associated with each forecast. Here, we report the width (or relative size) of the 95% CI derived from each method’s ensemble or state-space spread, indicating how tightly the model constrains its predictions.

The comparison metric is as follows:

Model	MSE	MAE	CI
SARIMA	4701.262392	43.663004	Maximum
TBATS	2751.344432	36.466887	Average
ANN	1859.273219	33.681535	Very narrow

Table 3.1: Comparison between time series models and neural networks

Model	Advantage	Disadvantage
SARIMA	Fast and highly interpretable via explicit AR/MA/seasonal terms.	Unable to model nonlinearities or evolving variance.
TBATS	Handles multiple and non-integer seasonal cycles flexibly.	Computationally intensive and less transparent.
ANN	Learns complex nonlinear patterns and provides data-driven CIs.	Black-box model that demands careful tuning and heavier training.

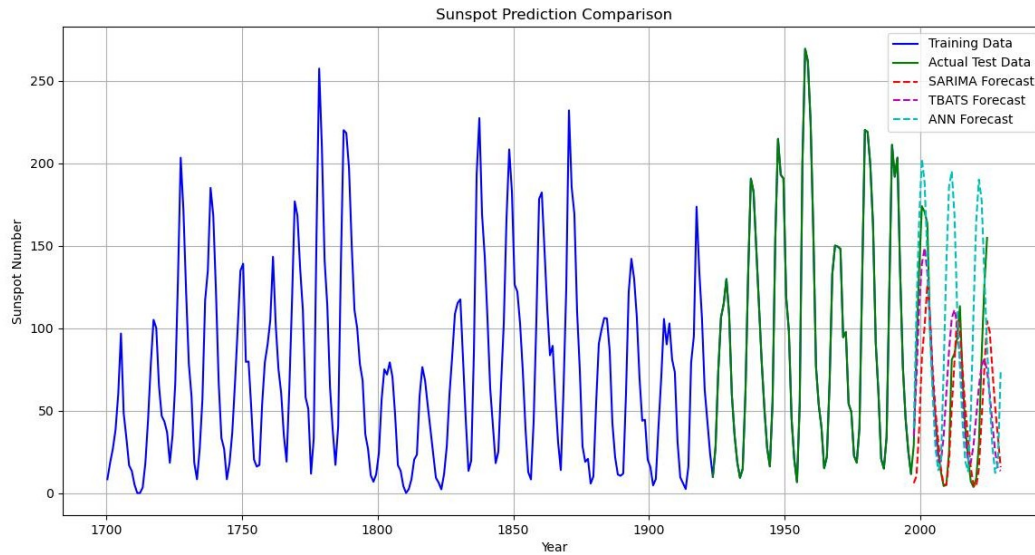


Figure 3.4: Comparison of observed sunspot numbers (1700–present) versus forecasts from SARIMA, TBATS, and ANN models.

Thus, from our figure, we see that:

- **SARIMA:** Underpredicts peak amplitudes due to rigid seasonal constraints.
- **TBATS:** Exhibits no significant amplitude bias but shows some phase lag during solar minima.
- **ANN:** Overfits to training noise.

FUTURE PROSPECTS

Our study demonstrates the use of SARIMA, TBATS and ANN in forecasting the sunspot number using past observations. While each method has shown promise in capturing various aspects of the time series, there remains substantial scope for further development and refinement of the models as well as more scope of investigation in the study of sunspots. In future, we plan to extend our work in the following directions:

1. Using hybrid models, that is, combining traditional time series prediction models with neural networks for forecasting.
2. Working with the daily and weekly sunspots data to get more precise results.
3. Studying other phenomena such as Coronal Mass Ejections and Solar Flares that occur during the Solar cycle.
4. Studying the effect of these solar activities in the form of Geomagnetic storm and its impact on Earth's atmosphere.

SARIMA code

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from statsmodels.tsa.statespace.sarimax import SARIMAX
5 from sklearn.metrics import mean_squared_error
6
7 # Load data
8 data_test = pd.read_csv(r"D:\ASTRO\sunspot prediction\SolarCycles.csv")
9 data_train = pd.read_csv(r"D:\ASTRO\sunspot prediction\SN_y_tot_V2.0.csv")
10
11 # Prepare series
12 ss_test = data_test["mean_sunspot_no"].values
13 year_test = data_test["Year"].values
14 ss_train = data_train["mean_sunspot_no"].values
15 year_train = data_train["Year"].values
16
17 # Fit SARIMA model
18 model = SARIMAX(ss_train,
19                 order=(1, 1, 1),          # Non-seasonal (p,d,q)
20                 seasonal_order=(1, 1, 1, 11), # Seasonal (P,D,Q,S) with
21                 11-year cycle
22                 enforce_stationarity=False,
23                 enforce_invertibility=False)
24
25 results = model.fit(dispatch=False)
26
27 # Forecast with confidence intervals
28 forecast_steps = 33
29 forecast_result = results.get_forecast(steps=forecast_steps)
30 forecast_values = forecast_result.predicted_mean
31 confidence_intervals = forecast_result.conf_int(alpha=0.05) # Returns
32                      DataFrame with lower/upper columns
33
34 # Generate forecast years (assuming last train year is 1996.5)
35 last_train_year = year_train[-1]
```

```
34 year_forecast = np.linspace(last_train_year + 1,
35                             last_train_year + forecast_steps,
36                             num=forecast_steps)
37
38 # Plot everything
39 plt.figure(figsize=(12, 6))
40 plt.plot(year_train, ss_train, label='Training Data')
41 plt.plot(year_test, ss_test, label='Actual Test Data')
42
43 # Plot confidence intervals
44 plt.fill_between(year_forecast,
45                 confidence_intervals[:, 0], # Lower bound
46                 confidence_intervals[:, 1], # Upper bound
47                 color='pink', alpha=0.3, label='95% CI')
48
49 plt.plot(year_forecast, forecast_values, label='SARIMA Forecast', color='red')
50 plt.xlabel("Year")
51 plt.ylabel("Sunspot Number")
52 plt.title("SARIMA Forecast of Yearly Sunspot Numbers")
53 plt.legend()
54 plt.grid(True)
55 plt.show()
```

TBATS code

```
1 import pandas as pd
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from tbats import TBATS
5 from sklearn.metrics import mean_squared_error
6
7 # Load data
8 data_test = pd.read_csv(r"D:\ASTRO\sunspot prediction\SolarCycles.csv")
9 data_train = pd.read_csv(r"D:\ASTRO\sunspot prediction\SN_y_tot_V2.0.csv")
10
11 # Prepare series
12 ss_test = data_test["mean_sunspot_no"].values
13 year_test = data_test["Year"].values
14 ss_train = data_train["mean_sunspot_no"].values
15 year_train = data_train["Year"].values
16
17 # Fit TBATS model
18 estimator = TBATS(seasonal_periods=[11], # 11-year cycle for yearly data
19                  use_box_cox=True,
20                  use_trend=True,
21                  use_damped_trend=True)
22
23 model = estimator.fit(ss_train)
24
25 # Force forecast with confidence intervals
26 forecast_steps = 33
27 forecast_values, confidence_dict = model.forecast(steps=forecast_steps,
28          confidence_level=0.95)
29
30 # Generate forecast years (assuming last train year is 1996.5)
31 last_train_year = year_train[-1]
32 year_forecast = np.linspace(last_train_year + 1,
33          last_train_year + forecast_steps,
34          num=forecast_steps)
```

```
35 # Plot everything
36 plt.figure(figsize=(12, 6))
37 plt.plot(year_train, ss_train, label='Training Data')
38 plt.plot(year_test, ss_test, label='Actual Test Data')
39
40 # Plot confidence intervals - using dictionary keys
41 plt.fill_between(year_forecast,
42                 confidence_dict['lower_bound'], # Dictionary key for lower
43                 bound
44                 confidence_dict['upper_bound'], # Dictionary key for upper
45                 bound
46                 color='pink', alpha=0.3, label='95% CI')
47
48 plt.plot(year_forecast, forecast_values, label='Forecast', color='red')
49 plt.xlabel("Year")
50 plt.ylabel("Sunspot Number")
51 plt.title("TBATS Forecast of Yearly Sunspot Numbers")
52 plt.legend()
53 plt.grid(True)
54 plt.show()
```

APPENDIX *C*

ANN code

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import MinMaxScaler
4 from sklearn.neural_network import MLPRegressor
5 import matplotlib.pyplot as plt
6
7 # Loading training data
8 train_df = pd.read_csv('SN_y_tot_V2.0.csv')
9 years_train = train_df['Year'].values
10 data_train = train_df['mean_sunspot_no'].values.reshape(-1, 1)
11
12 # Loading test data
13 test_df = pd.read_csv('SolarCycles.csv')
14 years_test = test_df['Year'].values
15 data_test = test_df['mean_sunspot_no'].values
16
17 # Scaling training data
18 scaler = MinMaxScaler(feature_range=(0, 1))
19 train_scaled = scaler.fit_transform(data_train)
20
21 # Sequences for supervised learning
22 def create_sequences(data, seq_length):
23     X, y = [], []
24     for i in range(len(data) - seq_length):
25         X.append(data[i : i + seq_length].flatten())
26         y.append(data[i + seq_length][0])
27     return np.array(X), np.array(y)
28
29 seq_length = 11 # length of input window (years)
30 X, y = create_sequences(train_scaled, seq_length)
31
32 # Train an ensemble of MLPRegressors and forecast next 33 years
33 n_models = 30
34 future_steps = 33 # forecast horizon in years
35 ensemble_preds = np.zeros((n_models, future_steps))
```

```

36
37 for i in range(n_models):
38     model = MLPRegressor(hidden_layer_sizes=(50,),
39                           activation='relu',
40                           solver='adam',
41                           max_iter=500,
42                           random_state=i)
43
44     model.fit(X, y)
45     seq = list(train_scaled[-seq_length:].flatten())
46     preds = []
47     for _ in range(future_steps):
48         inp = np.array(seq[-seq_length:]).reshape(1, -1)
49         p = model.predict(inp)[0]
50         preds.append(p)
51         seq.append(p)
52     ensemble_preds[i] = preds
53
54 # Inverse-transform the forecasts and compute 95% CI
55 forecast_mean = scaler.inverse_transform(
56     ensemble_preds.mean(axis=0).reshape(-1, 1)
57 ).flatten()
58 lower = scaler.inverse_transform(
59     np.percentile(ensemble_preds, 2.5, axis=0).reshape(-1, 1)
60 ).flatten()
61 upper = scaler.inverse_transform(
62     np.percentile(ensemble_preds, 97.5, axis=0).reshape(-1, 1)
63 ).flatten()
64 # Prepare the forecast timeline
65 last_year = int(years_train[-1])
66 future_years = np.arange(last_year + 1,
67                           last_year + 1 + future_steps)
68
69 # Plot training, test, and forecast series
70 plt.figure(figsize=(10, 5))
71 plt.plot(years_train, data_train,
72         label='Training Data', color='tab:blue')
73 plt.plot(years_test, data_test,
74         label='Test Data', color='tab:orange')
75 plt.plot(future_years, forecast_mean,
76         label='ANN Forecast', color='tab:red')
77 plt.fill_between(future_years,
78                 lower, upper,
79                 color='tab:red', alpha=0.3,
80                 label='95% CI')
81 plt.title('ANN Forecast of Sunspot Numbers')
82 plt.xlabel('Year')
83 plt.ylabel('Number of Sunspots')
84 plt.legend()
85 plt.grid(True, linestyle='--', alpha=0.5)
86 plt.show()

```

Bibliography

- [1] *The Sunspot Cycle, Solar Physics*. Marshall Space Flight Centre, NASA; Accessed on 18th June, 2025, from <https://solarscience.msfc.nasa.gov/SunspotCycle.shtml>
- [2] Galilei, G. *Istoria e Dimostrazioni intorno alle Macchie Solari*, 1613.
- [3] Schwabe, H., 1844, *Sonnen-Beobachtungen im Jahre 1843*, Astron. Nachr., 21(495), 233–236
- [4] Zhao et al., 2008, *Prediction of the Smoothed Monthly Mean Sunspot Numbers by Means of RBF (Radial Basic Function) Neural Networks*. Chinese Journal of Geophysics, 51(1), 20-24. <https://doi.org/10.1002/cjg2.1190>
- [5] Abdel-Rahman, H. I., & Marzouk, B. A. (2018). *Statistical method to predict the sunspots number*. NRIAG Journal of Astronomy and Geophysics, 7(2), 175–179. <https://doi.org/10.1016/j.nrjag.2018.08.001>
- [6] Pala, Z., Atici, R. *Forecasting Sunspot Time Series Using Deep Learning Methods*. Sol Phys 294, 50 (2019). <https://doi.org/10.1007/s11207-019-1434-6>
- [7] Solar Influences Data Analysis Center, The Royal Observatory of Belgium; Accessed on 18th June, 2025 from <https://www.sidc.be/SILSO/datafiles>
- [8] Solanki,Sami,K. *Sunspots: An Overview*, March,2003.
- [9] Luhmann, J.G.,Li,Y., Arge, C.N., Gazis ,P.R., Ulrich, R. *Solar cycle changes in coronal holes and space weather cycles*, Journal of Geophysical Research, Vol. 107, No. A8, 1154, 10.1029/2001JA007550, 2002.